Low-Level Design (LLD)

RAG-Based Customer Support Assistant

Version: 1.0
Date: April 2026
Author: Moin Uddin

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop Escalation

Table of Contents

1. Module-Level Design
2. Data Structures
3. Workflow Design (LangGraph)
4. Conditional Routing Logic
5. HITL Design
6. API / Interface Design
7. Error Handling

---

1. Module-Level Design

1.1 Document Processing Module (`src/document_processor.py`)

Responsibility: Load PDF files and split them into chunks suitable for embedding.

```
# Function Signatures

def load_pdf(pdf_path: str) -> list[Document]:
    """
    Load PDF and extract text page-by-page.
    Raises: FileNotFoundError, ValueError (empty PDF)
    """

def chunk_documents(
    documents: list[Document],
    chunk_size: int = 500,
    chunk_overlap: int = 100
) -> list[Document]:
    """
    Split documents using RecursiveCharacterTextSplitter.
    Adds chunk_index and total_chunks metadata.
    """

def process_knowledge_base(pdf_path: str = None) -> list[Document]:
    """
    End-to-end pipeline: load → chunk.
    If pdf_path is None, processes all PDFs in knowledge_base/ directory.
    """
```

Internal Flow:

```
load_pdf()
  ■■■ Open PDF with PyPDFLoader
  ■■■ Extract text per page → Document objects
  ■■■ Add source_file metadata
  ■■■ Return list[Document]

chunk_documents()
  ■■■ Initialize RecursiveCharacterTextSplitter
  ■    ■■■ chunk_size=500
  ■    ■■■ chunk_overlap=100
  ■    ■■■ separators=["\n\n", "\n", ". ", " ", ""]
```

```
■■■ Split documents
■■■ Add chunk_index and total_chunks metadata
■■■ Return list[Document]
```

1.2 Chunking Module (within Document Processing)

Strategy: RecursiveCharacterTextSplitter

Algorithm:

1. Try to split on `"\n\n"` (paragraph boundaries)
2. If resulting chunks > chunk_size, split on `"\n"` (line breaks)
3. If still too large, split on `". "` (sentence boundaries)
4. If still too large, split on `" "` (word boundaries)
5. Last resort: split on character boundaries

Parameters:

```
{
    "chunk_size": 500,        # Max characters per chunk
    "chunk_overlap": 100,     # Characters shared between consecutive chunks
    "length_function": len,   # Character-based measurement
    "add_start_index": True,  # Track position in original document
}
```

Why these values:

• **500 chars:** Approximately 100-125 words — enough to capture a complete FAQ answer or a meaningful paragraph, but granular enough for precise retrieval.

• **100 chars overlap (20%):** Prevents context loss at chunk boundaries. If a sentence spans two chunks, the overlap ensures it appears fully in at least one.

1.3 Embedding Module (`src/llm.py` - `get_embeddings()`)

```
def get_embeddings() -> HuggingFaceEmbeddings:
    """
    Initialize local HuggingFace embedding model.
    Model: all-MiniLM-L6-v2 (runs locally, no API needed)
    Output: 384-dimensional vectors
    """
```

Embedding Process:

```
Input Text (string) → HuggingFace Model (local) → 384-float vector

Example:
"How do I reset my password?" →
[0.0234, -0.0891, 0.1456, ..., 0.0023]  # 384 dimensions
```

1.4 Vector Storage Module (`src/vector_store.py`)

```
def initialize_vectorstore(chunks: list, embeddings) -> Chroma:
```

```
    """
    Create new ChromaDB collection.
    Clears existing data first to prevent duplicates.
    Persists to: ./chroma_db/
    Collection: "customer_support_docs"
    """

def get_vectorstore(embeddings) -> Chroma:
    """
    Load existing ChromaDB collection.
    Raises: FileNotFoundError, ValueError (empty collection)
    """

def clear_vectorstore() -> None:
    """Delete the chroma_db/ directory."""

def get_collection_stats(embeddings) -> dict:
    """Return collection status, document count, and paths."""
```

ChromaDB Configuration:

```
{
    "persist_directory": "./chroma_db/",
    "collection_name": "customer_support_docs",
    "distance_function": "cosine",  # Default for Chroma
}
```

1.5 Retrieval Module (`src/retriever.py`)

```
def create_retriever(vectorstore, k: int = 3):
    """Create LangChain retriever with similarity search."""

def retrieve_with_scores(vectorstore, query: str, k: int = 3) -> list[tuple]:
    """
    Return: [(Document, score), ...] sorted by relevance.
    Score range: 0.0 (irrelevant) to 1.0 (exact match)
    """

def check_retrieval_confidence(results: list) -> dict:
    """
    Evaluate retrieval quality.
    Returns confidence_level, scores, is_sufficient, needs_escalation.
    """

def format_retrieved_context(results: list) -> str:
    """Format docs with source attribution for LLM prompt."""
```

Confidence Evaluation Logic:

```
if top_score >= 0.5:    → HIGH confidence   → Proceed to generation
if top_score >= 0.3:    → MEDIUM confidence → Proceed to generation (with caution)
if top_score < 0.3:     → LOW confidence    → ESCALATE to human
if results == empty:    → LOW confidence    → ESCALATE to human
```

1.6 Query Processing Module (`src/intent_classifier.py`)

```
def classify_intent(query: str, llm) -> dict:
    """
    Classify query into one of 6 categories using LLM.
    Returns: {intent, needs_escalation, confidence}
    """
```

```
def _fuzzy_match_intent(raw_intent: str) -> str:
    """
    Fallback keyword matching when LLM output doesn't match categories.
    """

def get_escalation_reason(intent: str, query: str) -> str:
    """Generate human-readable escalation reason."""
```

Intent Categories and Routing:

```
GENERAL_QUERY    → Proceed to retrieval → Generate response
TECHNICAL_ISSUE  → Proceed to retrieval → Generate response
BILLING          → Proceed to retrieval → Generate response
COMPLAINT        → IMMEDIATE ESCALATION (skip retrieval)
ESCALATE         → IMMEDIATE ESCALATION (skip retrieval)
OUT_OF_SCOPE     → IMMEDIATE ESCALATION (skip retrieval)
```

1.7 Graph Execution Module (`src/graph_workflow.py`)

```
def classify_intent_node(state: GraphState) -> dict:
    """Node 1: Classify intent, set escalation flag."""

def retrieve_documents_node(state: GraphState) -> dict:
    """Node 2: Retrieve documents from ChromaDB."""

def evaluate_confidence_node(state: GraphState) -> dict:
    """Node 3: Score retrieval quality."""

def generate_response_node(state: GraphState) -> dict:
    """Node 4: Generate LLM response with context."""

def escalate_to_human_node(state: GraphState) -> dict:
    """Node 5: Create escalation ticket + message."""

def route_after_intent(state: GraphState) -> str:
    """Conditional edge: intent-based routing."""

def route_after_confidence(state: GraphState) -> str:
    """Conditional edge: confidence-based routing."""

def build_graph() -> StateGraph:
    """Construct and compile the full workflow graph."""

def run_graph(query: str, chat_history: list = None) -> dict:
    """Execute the graph with a query, return final state."""
```

1.8 HITL Module (`src/hitl.py`)

```
def check_escalation_needed(state: dict) -> bool:
    """
    Multi-factor escalation check:
    1. Intent in ESCALATION_INTENTS
    2. Retrieval confidence < threshold
    3. Explicit needs_escalation flag
    """

def create_escalation_ticket(state: dict) -> dict:
    """
    Package context into ticket:
    {ticket_id, timestamp, query, intent, confidence,
     reason, retrieved_context, status}
```

```
    """

def integrate_human_response(state: dict, human_input: str) -> dict:
    """Merge human response into state, clear escalation flags."""

def get_escalation_message(state: dict) -> str:
    """Generate intent-specific user-facing escalation message."""
```

---

2. Data Structures

2.1 Document Representation

```
# LangChain Document (from langchain_core.documents)
class Document:
    page_content: str    # The actual text content
    metadata: dict       # Associated metadata

# Example after loading:
Document(
    page_content="TechFlow Solutions is a leading provider...",
    metadata={
        "source": "C:/path/to/customer_support_guide.pdf",
        "page": 0,
        "source_file": "customer_support_guide.pdf",
    }
)
```

2.2 Chunk Format

```
# Example after chunking:
Document(
    page_content="To create a new TechFlow account: 1) Visit app.techflow.com...",
    metadata={
        "source": "C:/path/to/customer_support_guide.pdf",
        "page": 2,
        "source_file": "customer_support_guide.pdf",
        "start_index": 1245,      # Position in original page
        "chunk_index": 7,         # Global chunk number
        "total_chunks": 42,       # Total chunks in knowledge base
    }
)
```

2.3 Embedding Structure

```
# Vector stored in ChromaDB:
{
    "id": "doc_7",                           # Unique chunk ID
    "embedding": [0.0234, -0.0891, ...],     # 384-dim float vector
    "document": "To create a new TechFlow...", # Raw text
    "metadata": {
        "source_file": "customer_support_guide.pdf",
        "page": 2,
        "chunk_index": 7,
    }
}
```

2.4 Query-Response Schema

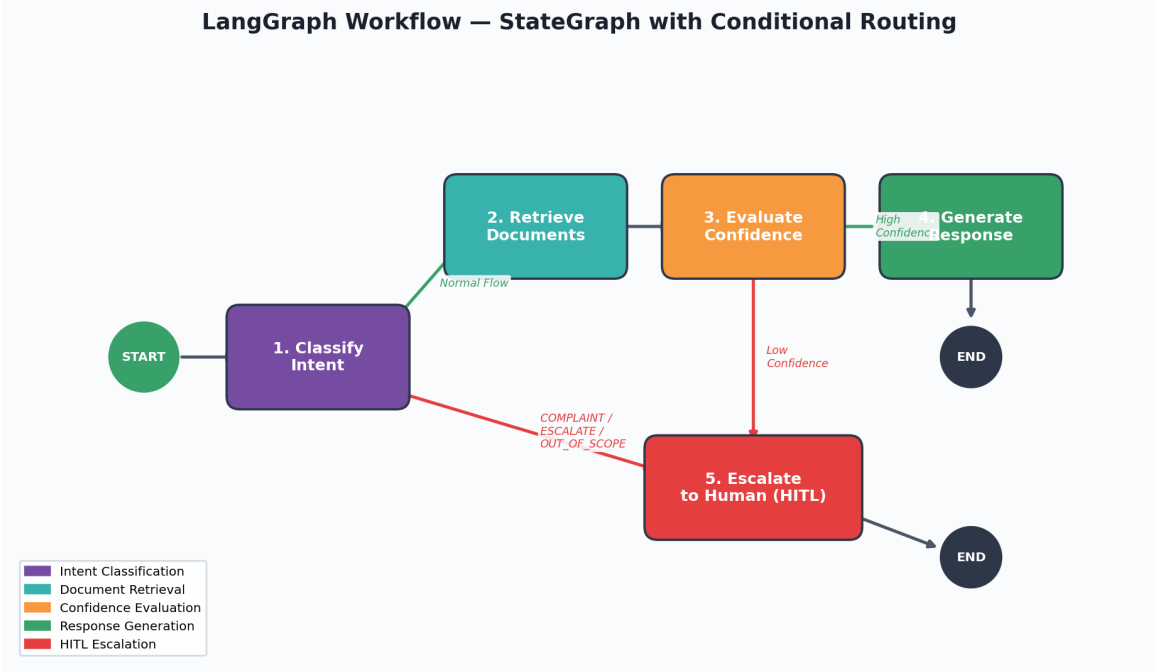
```
# Input to the system:
{
    "query": "How do I reset my password?",
    "chat_history": [
        {"role": "user", "content": "Hello"},
        {"role": "assistant", "content": "Hi! How can I help?"},
    ]
}

# Output from the system:
{
    "response": "To reset your password, follow these steps...",
    "intent": "GENERAL_QUERY",
    "sources": [
        {
            "content": "If you've forgotten your password...",
            "source": "customer_support_guide.pdf",
            "page": 2,
            "score": 0.892,
        }
    ],
    "needs_escalation": False,
    "retrieval_confidence": 0.892,
}
```

2.5 State Object for Graph (GraphState)

```
class GraphState(TypedDict):
    # Input
    query: str                          # User's raw query
    chat_history: list                  # Previous messages

    # Intent Classification
    intent: str                         # "GENERAL_QUERY" | "BILLING" | etc.
    intent_confidence: str              # "high" | "inferred" | "fallback"

    # Retrieval
    retrieved_docs: list                # [(Document, score), ...]
    context: str                        # Formatted context string
    sources: list                       # Source info for display

    # Confidence
    retrieval_confidence: float         # 0.0 to 1.0
    confidence_details: dict            # Full assessment

    # Response
    response: str                       # Final answer text

    # Escalation
    needs_escalation: bool              # HITL trigger flag
    escalation_reason: str              # Human-readable reason
    escalation_ticket: Optional[dict]   # Ticket data
    human_response: str                 # Agent's response text
```

---

3. Workflow Design (LangGraph)



3.1 Node Definitions

Node	Function	Input (from State)	Output (to State)
`classify_intent`	Classify query intent via LLM	`query`	`intent`, `intent_confidence`, `needs_escalation`, `escalation_reason`
`retrieve_documents`	Search ChromaDB for relevant chunks	`query`	`retrieved_docs`, `context`, `sources`
`evaluate_confidence`	Score retrieval quality	`retrieved_docs`	`retrieval_confidence`, `confidence_details`, `needs_escalation`
`generate_response`	Generate answer using LLM + context	`query`, `context`, `chat_history`	`response`
`escalate_to_human`	Create escalation ticket + message	`query`, `intent`, `retrieval_confidence`, `escalation_reason`	`response`, `escalation_ticket`, `needs_escalation`

3.2 Edge Definitions

From	To	Type	Condition
`START`	`classify_intent`	Entry Point	Always
`classify_intent`	`retrieve_documents`	Conditional	`needs_escalation == False`

classify_intent	escalate_to_human	Conditional	needs_escalation == True
retrieve_documents	evaluate_confidence	Direct	Always
evaluate_confidence	generate_response	Conditional	needs_escalation == False
evaluate_confidence	escalate_to_human	Conditional	needs_escalation == True
generate_response	END	Terminal	Always
escalate_to_human	END	Terminal	Always

3.3 State Transitions

```
State at START:
{query: "How to reset password?", intent: "", response: "", needs_escalation: False, ...}

After classify_intent:
{..., intent: "GENERAL_QUERY", intent_confidence: "high", needs_escalation: False}

After retrieve_documents:
{..., retrieved_docs: [(doc1, 0.89), (doc2, 0.76), (doc3, 0.71)], context: "...", sources: [...]}

After evaluate_confidence:
{..., retrieval_confidence: 0.89, confidence_details: {level: "high", ...}, needs_escalation: False}

After generate_response:
{..., response: "To reset your password, follow these steps: 1) Go to..."}

FINAL STATE → returned to caller
```

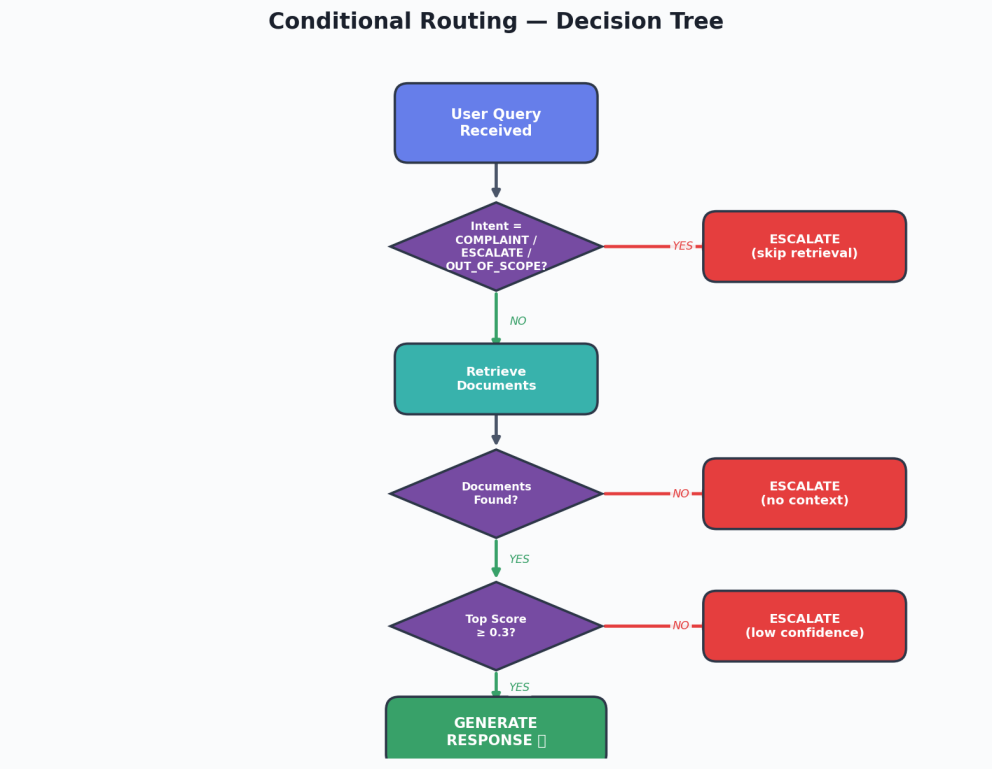
3.4 Graph Construction Code

```
workflow = StateGraph(GraphState)

# Add nodes
workflow.add_node("classify_intent", classify_intent_node)
workflow.add_node("retrieve_documents", retrieve_documents_node)
workflow.add_node("evaluate_confidence", evaluate_confidence_node)
workflow.add_node("generate_response", generate_response_node)
workflow.add_node("escalate_to_human", escalate_to_human_node)

# Set entry point
workflow.set_entry_point("classify_intent")

# Conditional edges
workflow.add_conditional_edges("classify_intent", route_after_intent,
    {"escalate_to_human": "escalate_to_human", "retrieve_documents": "retrieve_documents"})

workflow.add_edge("retrieve_documents", "evaluate_confidence")

workflow.add_conditional_edges("evaluate_confidence", route_after_confidence,
    {"escalate_to_human": "escalate_to_human", "generate_response": "generate_response"})

# Terminal edges
workflow.add_edge("generate_response", END)
workflow.add_edge("escalate_to_human", END)
```

```
graph = workflow.compile()
```

---

4. Conditional Routing Logic



4.1 Answer Generation Criteria

A response is generated automatically when ALL of the following are true:

```
def should_generate_response(state):
    return (
        state["intent"] not in ["COMPLAINT", "ESCALATE", "OUT_OF_SCOPE"]
        and state["retrieval_confidence"] >= 0.3
        and state["needs_escalation"] is False
        and len(state["retrieved_docs"]) > 0
    )
```

4.2 Escalation Criteria

Escalation is triggered when ANY of the following conditions is true:

4.2.1 Low Confidence

```
# Triggered in evaluate_confidence_node
if top_retrieval_score < 0.3:
    needs_escalation = True
    reason = f"Low retrieval confidence (score: {top_score:.3f})"
```

4.2.2 Missing Context

```
# Triggered in retrieve_documents_node
if len(retrieved_docs) == 0:
    needs_escalation = True
    reason = "No relevant documents found in knowledge base"

# Also triggered when vector store is not initialized
if not os.path.exists(CHROMA_PERSIST_DIR):
    needs_escalation = True
    reason = "Knowledge base not initialized"
```

4.2.3 Complex Query (Intent-Based)

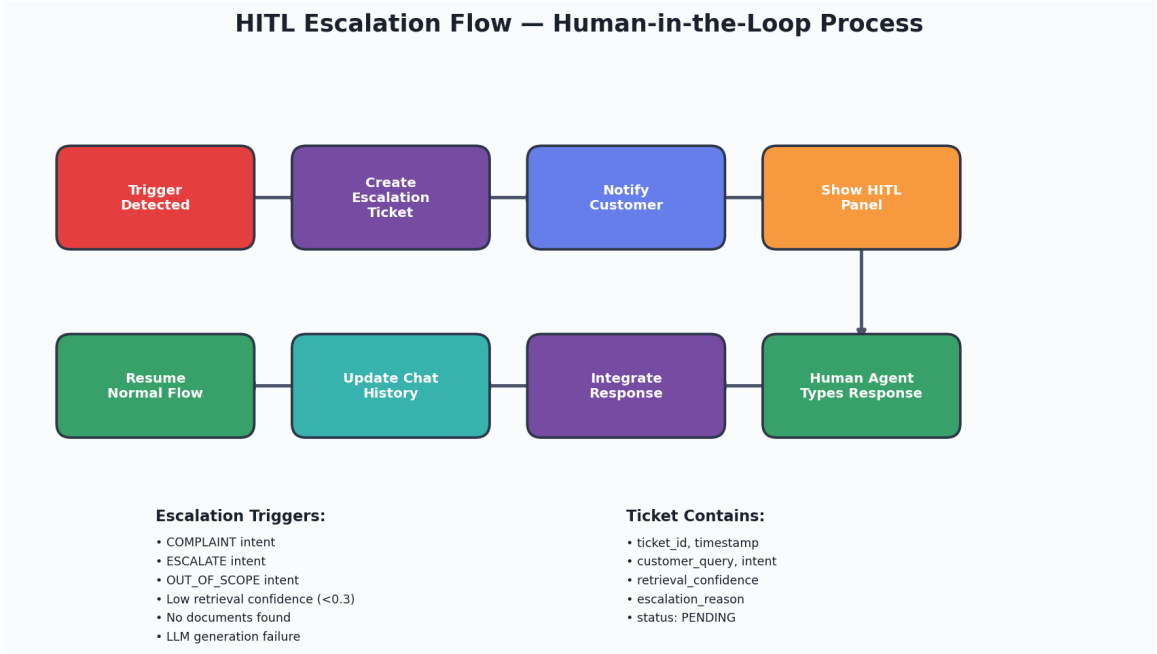
```
# Triggered in classify_intent_node
ESCALATION_INTENTS = ["COMPLAINT", "ESCALATE", "OUT_OF_SCOPE"]

if intent in ESCALATION_INTENTS:
    needs_escalation = True
    reason = get_escalation_reason(intent, query)
```

4.3 Decision Tree

```
Query Received
    ■
    ▼
Is intent COMPLAINT/ESCALATE/OUT_OF_SCOPE?
    ■■■ YES → ESCALATE (skip retrieval entirely)
    ■
    ■■■ NO → Retrieve documents
                ■
                ▼
            Are any documents retrieved?
                ■■■ NO → ESCALATE (no context)
                ■
                ■■■ YES → Evaluate confidence
                            ■
                            ▼
                        Is top_score >= 0.3?
                            ■■■ NO → ESCALATE (low confidence)
                            ■
                            ■■■ YES → GENERATE RESPONSE
```

---

5. HITL Design



5.1 When Escalation is Triggered

Trigger	Condition	Priority
Explicit Request	Intent == "ESCALATE"	Immediate
Customer Complaint	Intent == "COMPLAINT"	Immediate
Out-of-Scope Query	Intent == "OUT_OF_SCOPE"	Immediate
Low Confidence	top_retrieval_score < 0.4	After retrieval
No Documents Found	len(retrieved_docs) == 0	After retrieval
LLM Failure	Exception during generation	After generation attempt
KB Not Initialized	Vector store missing	Before retrieval

5.2 What Happens After Escalation

Step-by-step process:

1. **Trigger Detection:** One of the conditions above is met during graph execution.
2. **Ticket Creation:** `create_escalation_ticket()` packages all context:

• Customer's query
• Detected intent
• Retrieval confidence score
• Any retrieved documents (for human reference)
• Escalation reason

3. **User Notification:** A user-facing message explains that the query is being transferred.
4. **HITL Interface:** In the Streamlit UI, an escalation panel appears with:

• The context summary

- A text area for the human agent to type a response
- Send / Skip buttons

5. **Graph Pause:** The graph execution completes with `needs_escalation=True` in the final state.

5.3 How Human Response is Integrated

```
def integrate_human_response(state: dict, human_input: str) -> dict:
    """
    1. Takes the human agent's typed response
    2. Wraps it with a "[Human Agent]" prefix
    3. Updates the state:
       - response = formatted human response
       - needs_escalation = False (resolved)
       - escalation_reason = "" (cleared)
    4. Returns updated state for chat history
    """
```

In the UI:

```
User: "I want to cancel my account and I'm very frustrated!"
    ■
    ▼
Bot: "I'm sorry to hear about your experience. I'm escalating
      this to a human agent who can better assist you."
    ■
    ▼
[HITL Panel Appears]
Human Agent types: "I understand your frustration. Let me help
you with the cancellation. I've initiated the process and your
account will remain active until the end of this billing cycle."
    ■
    ▼
Bot: "[This response was provided by a human support agent]
      I understand your frustration. Let me help you..."
```

---

6. API / Interface Design

6.1 Input Format

Primary interface (Streamlit / CLI):

```
# User provides a natural language query string
input: str = "How do I reset my password?"

# System internally builds:
{
    "query": "How do I reset my password?",
    "chat_history": [
        {"role": "user", "content": "previous question"},
        {"role": "assistant", "content": "previous answer"},
    ]
}
```

6.2 Output Format

```
# run_graph() returns:
{
    "query": str,                     # Original query
    "intent": str,                    # Classified intent
    "response": str,                  # Answer or escalation message
    "sources": [                      # Documents used
        {
            "content": str,           # Chunk text (truncated)
            "source": str,            # Filename
            "page": int,              # Page number
            "score": float,           # Relevance score
        }
    ],
    "needs_escalation": bool,         # Was it escalated?
    "escalation_reason": str,         # Why (if escalated)
    "retrieval_confidence": float,    # Top score
    "escalation_ticket": dict|None,   # Ticket data (if escalated)
}
```

6.3 Interaction Flow

```
■■■
```

Vector store missing	`get_vectorstore()`	Raise `FileNotFoundError`, UI shows upload prompt
Empty collection	`get_vectorstore()`	Raise `ValueError`, suggest re-ingestion
No relevant chunks	`retrieve_with_scores()`	Return empty list → confidence evaluator escalates
LLM API failure	`generate_response_node()`	Catch exception → escalate with error message
Intent parse error	`classify_intent()`	Fallback to `GENERAL_QUERY` intent
Invalid intent	`_fuzzy_match_intent()`	Keyword-based fallback matching
Network error	Any API call	Caught at node level → escalation with error context
ChromaDB corruption	`get_vectorstore()`	Clear and re-ingest

7.2 Error Flow

```
# Example: LLM failure in generate_response_node
try:
    response = llm.invoke(messages)
    return {"response": response.content}
except Exception as e:
    return {
        "response": "I apologize, but I encountered an error...",
        "needs_escalation": True,
        "escalation_reason": f"LLM generation error: {str(e)}",
    }
```

7.3 Graceful Degradation Chain

```
Normal Flow → [Error] → Retry? → [Error] → Escalate to Human → [Error] → Show Error Message
```

The system never crashes silently. Every error path leads to either:

1. A user-facing error message
2. An escalation to a human agent
3. A prompt to re-initialize the system

End of Low-Level Design Document